# Image Stitching in CudaSift

## How a RANSAC homography becomes two aligned images on a shared canvas

CudaSift 2.4.0 (rct-scientific-proc fork), commit 1c26eba. Figures and numbers were produced by running the built library (build/Release/cusift.dll) on an NVIDIA GeForce RTX 4070 on 2 November 2026.

**Scope.** This document follows the data from a set of matched SIFT features to the pair of warped images that the library returns. It covers the RANSAC homography estimator and its host-side refinement, the way the output canvas is sized from the homography, the inverse-mapping warp kernel, and what the caller is expected to do afterwards. Feature extraction and descriptor matching are summarised only as far as they shape the inputs of the geometry stages. Every statement is tied to a function in the source tree; a code map at the end lists file and line for each.

## Contents

**1. Overview . . . 3**

**2. Conventions used throughout the code . . . 3**

**3. Inputs to the geometry: keypoints and matches . . . 4**

**4. Estimating the homography with RANSAC on the GPU . . . 4**

4.1 Candidate filtering and sampling . . . 4

4.2 One hypothesis per thread: the 4-point DLT . . . 5

4.3 Scoring every hypothesis against every match . . . 5

4.4 Picking the winner . . . 5

4.5 The similarity alternative . . . 5

**5. Refining the homography on the host . . . 6**

**6. From H to a canvas: sizing the output . . . 6**

**7. Warping by inverse mapping . . . 8**

7.1 The per-pixel computation . . . 8

7.2 Memory layout and the three entry points . . . 8

**8. What the caller does with the two canvases . . . 9**

**9. Worked example with a known answer . . . 9**

9.1 The bundled test pair, and what a homography cannot do . . . 12

**10. Multi-attempt estimation . . . 13**

**11. Failure modes and guards . . . 14**

**12. Code map . . . 15**

**13. Measured cost of each stage . . . 15**

**Appendix A. Minimal C usage of the stitch step . . . 16**

**Appendix B. The same step from Python . . . 16**

# 1. Overview

CudaSift does not produce a single blended panorama. Its stitching output is a pair of images, `warped_image1` and `warped_image2`, that share one coordinate frame and one pixel grid. Image 1 is copied into that frame unchanged; image 2 is resampled through the homography so that the same scene point lands on the same canvas pixel in both outputs. Pixels that no source covers are filled with NaN. Blending the two canvases into one picture is deliberately left to the caller, because the right blend depends on the application (a quick average, a feathered seam, exposure compensation, or no blend at all when the goal is change detection).

The pipeline has six stages. The first four produce and polish the 3×3 homography H; the last two consume it.

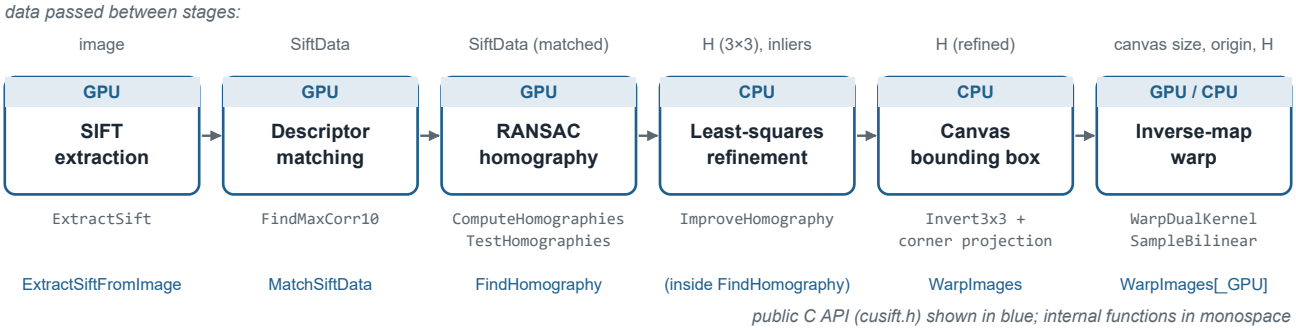



*Figure 1. The stitching pipeline. Each stage can be called on its own through the public C API, or fused through* `ExtractAndMatchAndFindHomographyAndWarp` *and its* `_GPU` *and* `_Multi_AndWarp` *variants.*

| Stage | Runs on | Input | Output | Public entry point |
|---|---|---|---|---|
| SIFT extraction | GPU | float32 grayscale image | `SiftData`: up to `max_keypoints_` keypoints with 128-d descriptors | `ExtractSiftFromImage` |
| Descriptor matching | GPU | two SiftData | for every keypoint of image 1: `match`, `match_xpos`, `match_ypos`, `score`, `ambiguity` | `MatchSiftData` |
| RANSAC homography | GPU | matched SiftData of image 1 | best 4-point hypothesis H, inlier count | `FindHomography` |
| Refinement | CPU (AVX2) | H and the matched points | least-squares H over all inliers; per-point match_error | `FindHomography` |
| Canvas sizing | CPU | H, both image sizes | canvas width, height, origin (u0, v0), corner positions | `WarpImages`, `ComputeWarpGeometry` |
| Warp | GPU or CPU | both images, H, canvas | two canvases, NaN where empty | `WarpImages`, `WarpImages_GPU` |

# 2. Conventions used throughout the code

- **Pixel coordinates** are 0-based, x along columns and y along rows, origin at the top-left pixel. Images are row-major float32 buffers in the range [0, 255] (`Image_t`), or pitched device buffers whose row stride in bytes is carried separately (`ImageStrided_t`).

- **The homography maps image 1 to image 2.** For a point (x1, y1) in image 1, H·(x1, y1, 1) is proportional to (x2, y2, 1) in image 2. It is stored row-major as nine floats and normalised so that the last element h22 equals 1; RANSAC and the refinement solve for the other eight.
- **All geometry works on the SiftData of image 1.** After matching, each `SiftPoint` of image 1 carries its own position (`xpos`, `ypos`), the position of its matched keypoint in image 2 (`match_xpos`, `match_ypos`), the match `score` and `ambiguity`, and later the reprojection `match_error`. The SiftData of image 2 is not touched again after matching.
- **The canvas lives in the frame of image 1.** Canvas pixel (x, y) corresponds to image-1 coordinate (x + u0, y + v0); u0 and v0 are integers and are usually zero or negative.

$$(x2', y2', z)^T = H \cdot (x1, y1, 1)^T, \quad x2 = x2' / z, \quad y2 = y2' / z, \quad z = h20 \cdot x1 + h21 \cdot y1 + 1$$

# 3. Inputs to the geometry: keypoints and matches

`ExtractSift` builds a Difference-of-Gaussians pyramid on the GPU, keeps extrema above the contrast threshold `thresh_` that pass the edge test, and computes a unit-length 128-d descriptor for each. The geometry stages only need the keypoint positions and the match fields, so the rest of extraction is out of scope here.

`FindMaxCorr10` (src/matching.cu:39) performs brute-force matching: every descriptor of image 1 is compared with every descriptor of image 2 by dot product, in 32×32 tiles staged through shared memory. Because descriptors are unit length the dot product is a cosine similarity, so a larger score is a better match. For each keypoint of image 1 the kernel keeps the best and the second-best score and records:

- `score` = best dot product;
- `ambiguity` = second-best / best. Values near 1 mean the match is not distinctive (Lowe's ratio test, stored rather than applied);
- `match`, `match_xpos`, `match_ypos` = index and position of the best candidate in image 2.

Note that **every** keypoint of image 1 receives a nearest neighbour; nothing is rejected at this stage. In the runs below, image 1 had 4005 keypoints and exactly 4005 matches. The ratio test is applied later, through `max_ambiguity_` (RANSAC candidates) and `improve_max_ambiguity_` (refinement). The C demo uses 1.0 for both, which accepts every match and relies on RANSAC alone; the Python defaults use 0.8, which in the bundled pair keeps only 158 of 4005 matches.

# 4. Estimating the homography with RANSAC on the GPU

`FindHomography` (src/cusift.cu:276) validates its options, dispatches to the selected model through `FindGeometricModel`, rejects a non-finite result, and then calls the host refinement. The projective model is implemented by `FindHomography_private` (src/matching.cu:408).

## 4.1 Candidate filtering and sampling

The scores and ambiguities of all points are copied to the host, and a point is a candidate when `score > min_score_` and `ambiguity < max_ambiguity_`. At least 8 candidates are required (and at least 8 points overall); otherwise the homography is left as NaN, which the public wrapper turns into the error "Homography contains non-finite values".

`GenerateRandomSamples` draws, for each of `num_loops_` hypotheses, four *distinct* candidate indices from a `std::mt19937` seeded with `seed_` (0 selects `std::random_device`). A fixed seed therefore gives a bit-for-bit repeatable estimate. `num_loops_` is rounded up to a multiple of 16 and capped at 65535×16 = 1 048 560, because the test kernel places 16 hypotheses per block along grid.y.

The four coordinate columns are copied device-to-device out of the SiftPoint array into four planar arrays [x1 | y1 | x2 | y2], each padded to a multiple of 16 entries. The padding is filled with NaN so that padded entries fail every inlier comparison instead of adding noise to the counts.

## 4.2 One hypothesis per thread: the 4-point DLT

`ComputeHomographies` (src/matching.cu:279) runs one thread per hypothesis. With h22 fixed to 1, each correspondence (x1, y1) → (x2, y2) gives two linear equations in the remaining eight unknowns, so four correspondences give the square 8×8 system A·h = b:

```
row 2i   : [ x1  y1  1   0   0   0   -x2*x1  -x2*y1 ] . h = x2
row 2i+1 : [ 0   0   0   x1  y1  1   -y2*x1  -y2*y1 ] . h = y2
```

The thread inverts A with a per-thread LU decomposition with partial pivoting (`InvertMatrix<8>`) and multiplies by b. The eight parameters are written plane-major (parameter j of hypothesis i at j·numLoops + i) so that the test kernel can read them with coalesced loads. Degenerate samples (three collinear points, duplicated locations) produce a wild or non-finite hypothesis; it is simply out-voted in the next step.

## 4.3 Scoring every hypothesis against every match

`TestHomographies` (src/matching.cu:329) uses blocks of 16×16 threads: 16 hypotheses per block, and 16 threads that stride over the correspondences for each hypothesis. The reprojection residual is evaluated in homogeneous form, which avoids one division per point and keeps the comparison exact when z is not 1:

$$z = h20\cdot x1 + h21\cdot y1 + 1, \quad e_x = x2\cdot z - (h00\cdot x1 + h01\cdot y1 + h02), \quad e_y = y2\cdot z - (h10\cdot x1 + h11\cdot y1 + h12)$$

$$\text{inlier if and only if } e_x^2 + e_y^2 < \text{thresh_}^2 \cdot z^2$$

The per-thread counts are reduced in shared memory and one integer per hypothesis is written back. In version 2.3.0 the residual is computed with round-to-nearest fused multiply-adds; the previous round-toward-zero intrinsics shrank the projection magnitude and biased the decision.

## 4.4 Picking the winner

The host copies the counts back, scans for the maximum, and reads the eight parameters of that hypothesis into `homography[0..7]`, with `homography[8] = 1`. The count becomes `num_matches`. This is the number of matches within `thresh_` pixels of the *unrefined* 4-point hypothesis; see section 5 for how it relates to the refined fit.

## 4.5 The similarity alternative

With `model_type_ = CUSIFT_MODEL_SIMILARITY`, `FindSimilarity_private` runs the same RANSAC skeleton with two-point samples and a closed-form 4-DOF solution [a, b, tx, ty] (rotation, uniform scale, translation). The winner is packed into the same 3×3 layout so that every downstream stage, including the warp, is unchanged:

```
[  a  -b  tx ]
[  b   a  ty ]      a = s*cos(theta),  b = s*sin(theta)
[  0   0   1 ]
```

Two-point samples are far more likely to be all-inlier than four-point samples, so this model converges with fewer loops and survives a lower inlier ratio. Use it when the camera motion is known to be free of shear and perspective; otherwise the residual perspective shows up as misalignment at the canvas edges.

## 5. Refining the homography on the host

The RANSAC winner is exact for its four sample points and only approximately right for the rest. `ImproveHomography` (src/geomFuncs.c:210) replaces it with a least-squares fit over all inliers, using iteratively re-weighted least squares (IRLS) with binary weights. Starting from A = H / h22, each of the `improve_num_loops_` rounds does the following for every point that passes `improve_min_score_` and `improve_max_ambiguity_`:

- project (xpos, ypos) through the current A and measure the squared distance err² to (match_xpos, match_ypos);
- set the weight w = 1 if err² ≤ `improve_thresh_`², else w = 0 (points with w = 0 are skipped entirely);
- accumulate the two DLT rows Y of that point into the normal equations M += w·Y·$Y^T$ and X += w·Y·m, where m is the measured x2 or y2;
- solve the 8×8 system M·A = X by Cholesky factorisation. If M is not positive definite the loop stops and the current A is kept.

The accumulation is the hot spot and is vectorised with AVX2 and FMA (eight-wide outer-product updates), with a portable scalar fallback selected at compile time. A build compiled for AVX2 checks at run time that the CPU supports it, and if not, prints a warning and returns without refining, leaving the RANSAC estimate in place.

After the last round a final pass writes the reprojection distance of every point into `match_error` and counts the points with error below `improve_thresh_`. That count is the function's return value, but the public `FindHomography` discards it: `num_matches` always reports the RANSAC count. In the bundled pair the two differ (1501 at thresh_ = 3 px versus 1468 at improve_thresh_ = 2 px); the per-point `match_error` field is the way to recover the refined inlier set, and the Python wrapper's `Keypoint.match_error` exposes it.

Why it matters for stitching: the warp is only as good as H at the image borders, far from the four sample points. In the bundled pair the median residual over the refined inliers is 0.37 px; in the synthetic example of section 9 it is 0.35 px.

## 6. From H to a canvas: sizing the output

`WarpImages` (src/cusift.cu:880), `WarpImages_GPU` and the fused pipeline functions all begin with the same two host-side steps. The canvas is defined in the frame of image 1, so the question is where image 2 lands in that frame. H maps image 1 to image 2, so the inverse is needed:

- **Invert H.** `Invert3x3` (src/cusift.cu:685) computes the inverse by cofactors; if |det H| < $10^{-12}$ the call fails with "Homography is too close to singular to invert".
- **Project the four corners of image 2**, (0, 0), (w2−1, 0), (w2−1, h2−1), (0, h2−1), through $H^{-1}$ with a homogeneous divide. These are the only four points pushed through the inverse; the warp itself never uses it.

- **Take the bounding box** of those four points together with the rectangle of image 1, [0, w1−1] × [0, h1−1], then round outward: u0 = floor(uMin), u1 = ceil(uMax), v0 = floor(vMin), v1 = ceil(vMax). The canvas is outW = u1 − u0 + 1 by outH = v1 − v0 + 1 pixels, and its origin offset is (originU, originV) = (u0, v0).
- **Guard the size.** If outW exceeds 2·max(w1, w2) or outH exceeds 2·max(h1, h2), the call fails with "warped image too large" rather than allocating a huge, mostly empty canvas from a bad H. The same 2× bound is what `EstimateVramWarpImages` assumes.

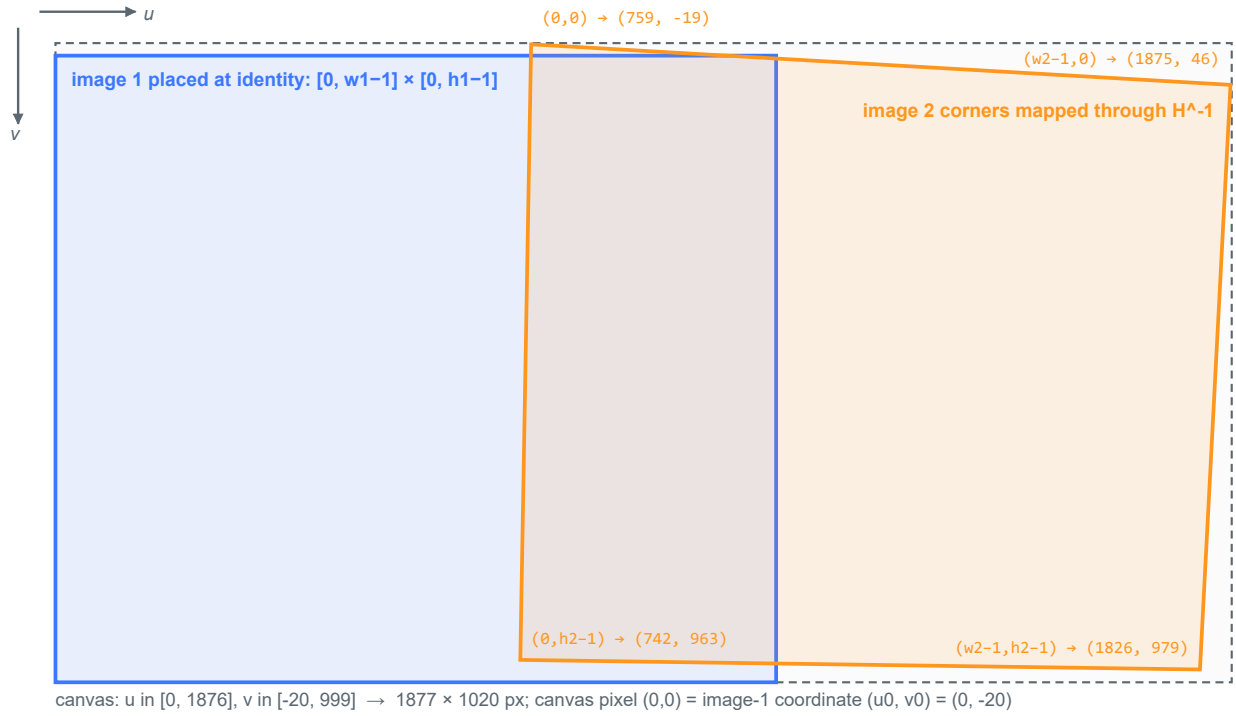



*Figure 2. Canvas construction for the synthetic pair of section 9 (image 1 is 1150×1000, image 2 is 1150×1000). The projected corners of image 2 extend past the right edge and above the top of image 1, so the canvas grows to 1877×1020 and its origin moves to v0 = -20. Drawn to scale from the numbers the library produced.*

Because u0 and v0 are integers, the canvas grid is the pixel grid of image 1 shifted by a whole number of pixels. Image 1 therefore lands exactly on canvas pixels, with no resampling blur, which is the reason the frame of image 1 was chosen as the reference.

All of this lives in one internal helper, `ComputeCanvasGeometry` (src/cusift.cu:723), which every warp entry point calls. The public `ComputeWarpGeometry` (src/cusift.cu:1071) exposes the same helper without warping: it fills a `WarpGeometry_t` with the canvas size, the origin (u0, v0) and the canvas coordinates of all four corners of each image, in constant time. Because both paths run the same code, the geometry a caller queries is exactly the geometry a subsequent warp uses; the regression suite checks this, together with a round trip of the returned corners through H.

# 7. Warping by inverse mapping

Both the GPU and CPU paths use *inverse mapping*: they iterate over output pixels and ask where each one comes from, rather than pushing source pixels forward. Forward mapping would leave holes where the transform stretches and double-write where it compresses; inverse mapping gives every canvas pixel exactly one value. The forward map is only used, on four points, to size the canvas in section 6.

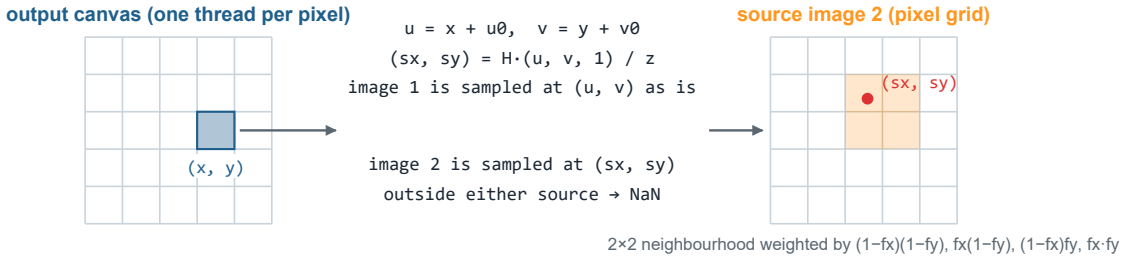



*Figure 3. One output pixel of the canvas, its location in image-1 coordinates, and the bilinear sample it pulls from image 2.*

## 7.1 The per-pixel computation

`WarpDualKernel` (src/cusift.cu:829) launches one thread per canvas pixel in 16×16 blocks and writes both outputs in a single pass:

```
u = x + originU;  v = y + originV;                      // canvas pixel -> image-1 coordinates

// image 1: identity.  (u, v) is integral, so this is an exact copy where in range.
val1 = (0 <= u < src1W-1 && 0 <= v < src1H-1) ? bilinear(src1, u, v) : NaN;

// image 2: forward H takes image-1 coordinates to image-2 pixel coordinates.
z  = h20*u + h21*v + h22;
sx = (h00*u + h01*v + h02) / z;
sy = (h10*u + h11*v + h12) / z;
val2 = (0 <= sx < src2W-1 && 0 <= sy < src2H-1) ? bilinear(src2, sx, sy) : NaN;
```

Bilinear sampling reads the 2×2 neighbourhood around the non-integer location and weights it by the fractional parts fx = sx − floor(sx), fy = sy − floor(sy):

$$\text{val} = (1-fx)(1-fy)\cdot I(x0, y0) + fx(1-fy)\cdot I(x0+1, y0) + (1-fx)fy\cdot I(x0, y0+1) + fx\cdot fy\cdot I(x0+1, y0+1)$$

Two consequences of the bounds test are worth knowing. First, it requires the whole 2×2 neighbourhood to be inside the source, so the last column and the last row of each source image are never sampled and come out as NaN even for image 1. Second, NaN is the library's "no data" marker: it is not a valid intensity, it renders as black when clamped to 8 bits, and callers can build an overlap mask with isnan() on the two canvases.

## 7.2 Memory layout and the three entry points

Device images are allocated with `cudaMallocPitch`, so every buffer carries its own row stride in elements. The kernel takes separate strides for its two sources and its two destinations; the 2.3.0 fix K6 made the second destination stride explicit instead of reusing the first. The three public routes differ only in where the data lives:

| Entry point | Source upload | Output | Caller frees with |
|---|---|---|---|
| `WarpImages(..., useGPU=true)` | both images uploaded to pitched device buffers | dense host buffers in two `Image_t` (copied back with cudaMemcpy2D) | `FreeImage` |
| `WarpImages(..., useGPU=false)` | none; OpenMP loop over canvas rows on the host, same arithmetic via SampleBilinear | dense host buffers | `FreeImage` |
| `WarpImages_GPU` | both images uploaded | pitched device buffers in two `ImageStrided_t` (stride_ in bytes) | `FreeImage_GPU` |
| `ExtractAndMatchAndFindHomographyAndWarp[_GPU]` | none extra: the pitched device images already uploaded for SIFT extraction are reused as warp sources | as above, host or device | `FreeImage` / `FreeImage_GPU` |

Measured on the bundled 1920×1080 pair (second run, after warm-up, timed from Python around the C calls): the GPU warp took 9.4 ms including both uploads and both downloads, and the CPU path took 22.2 ms. The fused pipeline skips the uploads entirely.

## 8. What the caller does with the two canvases

The library stops at alignment. A stitched picture is any per-pixel combination of the two canvases that respects the NaN mask. The figures in this document use the simplest rule: where only one source is valid take it, where both are valid take the mean. In NumPy:

```
va, vb = ~np.isnan(w1), ~np.isnan(w2)       # validity masks
out = np.full_like(w1, np.nan)
out[va & ~vb] = w1[va & ~vb]
out[vb & ~va] = w2[vb & ~va]
both = va & vb
out[both] = 0.5 * (w1[both] + w2[both])      # or: w1[both] for a hard seam, or a feathered weight
```

Common refinements, none of which need anything more from the library: prefer one image in the overlap and feather the weight with the distance to that image's border to hide the seam; equalise mean intensity in the overlap before blending to hide exposure differences; or skip blending and difference the two canvases to detect what changed. The Python package (`cusift_py/test.py`) writes the two canvases to disk as `warped_i1.png` and `warped_i2.png` and leaves compositing to the user in the same way.

Ownership is simple: `WarpImages` allocates the host buffers with malloc and `FreeImage` releases them; `WarpImages_GPU` hands back device memory that `FreeImage_GPU` (or cudaFree) releases. The Python wrapper copies the pixels into NumPy arrays and frees the C buffers before returning.

## 9. Worked example with a known answer

To show the canvas logic on a pair with real parallax, a second pair was cut from `test/img1.png`. Image A is the plain 1150×1000 crop at column 0, row 40. Image B is the right-hand part of the same picture viewed through a known transform: a 2° rotation, a 0.96 scale and a mild perspective term, rendered with bicubic resampling. The ground-truth homography from A to B follows from the construction, so the estimate can be checked.




*Figure 4. The synthetic pair: image A (left) and image B (right). They overlap in roughly the right third of A.*

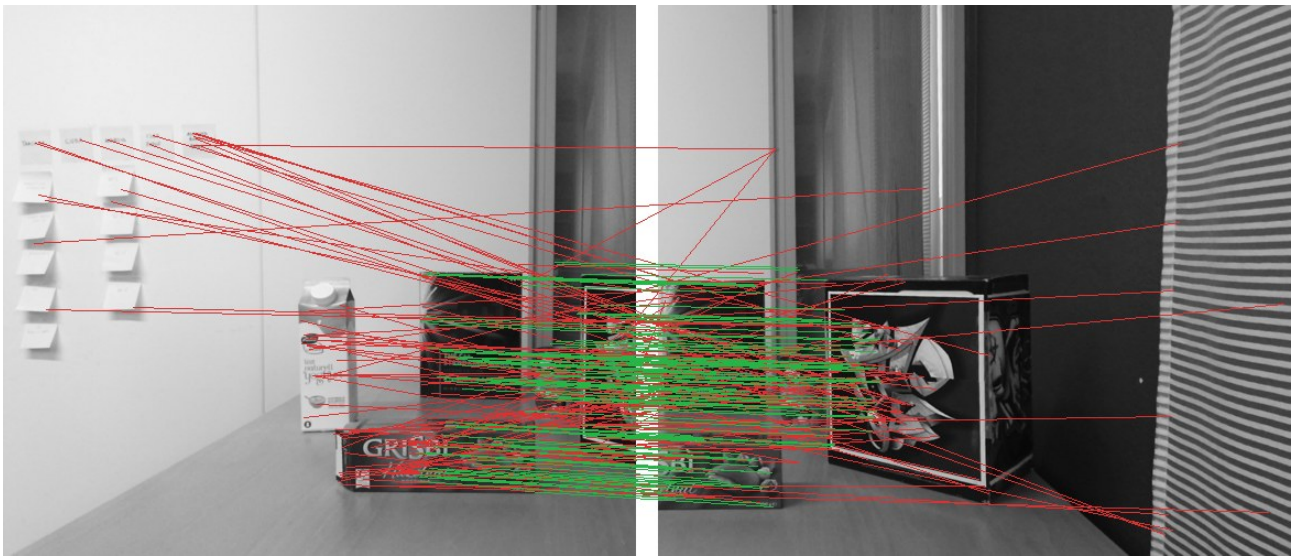


*Figure 5. A random 120 of the 831 refined inliers (green) and 120 of the 931 remaining matches (red). Every keypoint of A got a nearest neighbour in B; the ones outside the overlap are necessarily wrong and RANSAC rejects them.*

| Quantity | Value |
|---|---|
| keypoints in A / in B | 1762 / 3454 |
| matches (one per keypoint of A) | 1762 |
| RANSAC inliers at thresh_ = 3 px (reported num_matches) | 871 |
| refined inliers at improve_thresh_ = 2 px | 831 (median residual 0.35 px) |
| corners of B in the frame of A | (759.0, -19.1); (1875.1, 45.6); (1826.3, 978.6); (741.8, 963.3) |
| canvas | u in [0, 1876], v in [-20, 999] → 1877×1020 px, origin (0, -20) |
| canvas pixels: both / only A / only B / empty | 384,852 / 762,999 / 670,151 / 96,538 |

| Quantity | Value |
|---|---|
| max $\lvert H - H_{true} \rvert$ entry | 3.24 (translation entries; see text) |
| corner error of A under H vs $H_{true}$ | 3.86 px, 0.69 px, 0.56 px, 3.55 px |

The two matrices (estimated on the left, ground truth on the right):

$$\begin{bmatrix} 0.94709 & 0.01663 & -718.55048 \\ -0.05530 & 0.95457 & 60.16744 \\ -4.15e\text{-}05 & -3.08e\text{-}05 & 1.00000 \end{bmatrix} \qquad \begin{bmatrix} 0.95143 & 0.01642 & -721.79091 \\ -0.05294 & 0.95795 & 58.07634 \\ -3.79e\text{-}05 & -3.06e\text{-}05 & 1.00000 \end{bmatrix}$$

The entries differ most in the translation column, but matrix entries are not the right yardstick for a homography: where the transform sends points is. The corner errors tell the real story. The two corners of A inside or near the overlap are reproduced to within 0.7 px. The two far corners of A sit 700 px outside any match, and extrapolating the perspective row that far amplifies a sub-pixel fit to about 4 px. This is inherent to the problem, not to the estimator: a stitch is well-determined only where the images overlap.

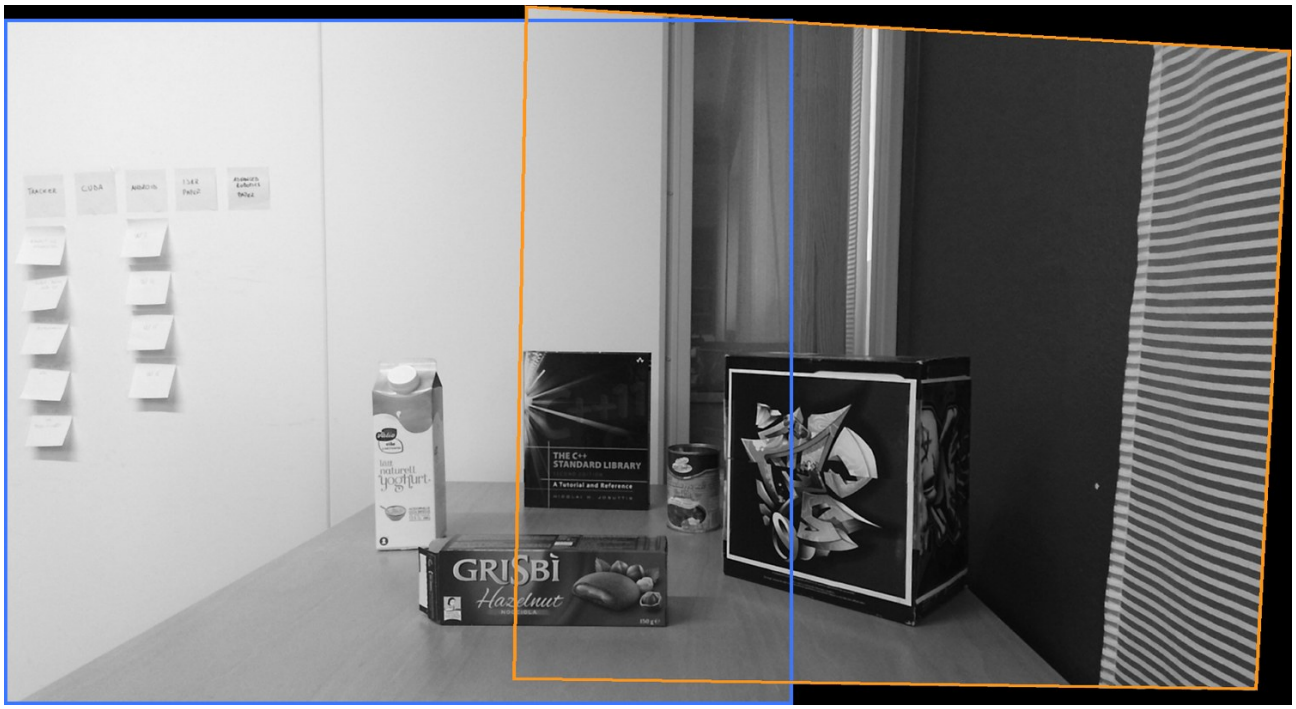



*Figure 6. The two canvases composited by the rule of section 8, with the footprint of image A (blue) and the projected footprint of image B (orange) drawn on top. Compare with Figure 2.*




*Figure 7. Alignment check: canvas of A in the red channel, canvas of B in green and blue. Where both are valid and aligned the result is neutral grey; misalignment would show as coloured fringes. Pure red and cyan mark regions covered by only one source.*

## 9.1 The bundled test pair, and what a homography cannot do

The repository's own pair, `test/img1.png` and `test/img2.png`, is a static camera with objects moved between the two shots. The estimate is close to the identity (a shift of about 0.22 and 0.72 px), the canvas is 1922×1081, and 1501 of 4005 matches are RANSAC inliers. Those inliers all lie on the wall, the window and the table; the matches on the moved boxes (the red lines in Figure 8) are consistent with each other but not with the background, and RANSAC votes with the majority.

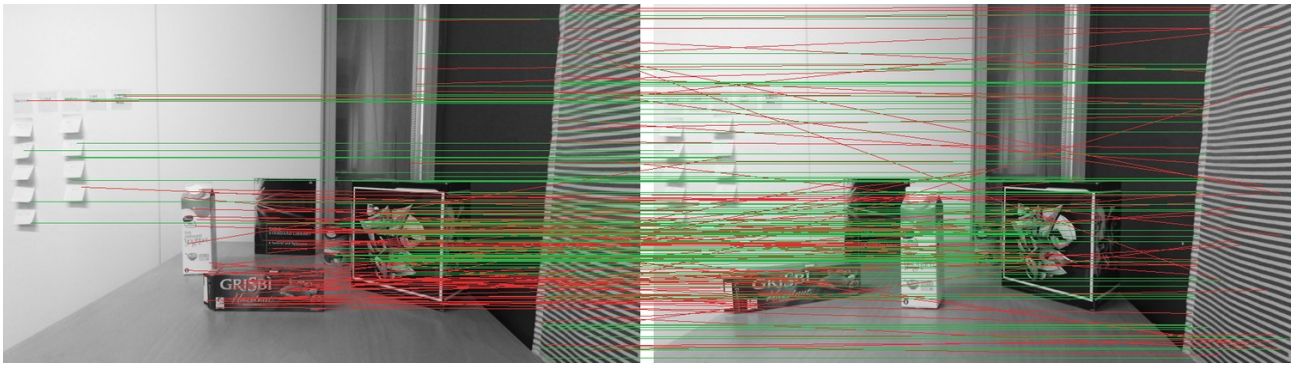


*Figure 8. The bundled pair. Green: inliers of the refined homography, which lie on the static background. Red: matches on the objects that moved between the shots.*




*Figure 9. Averaging the two canvases of the bundled pair. The background aligns to sub-pixel accuracy; the moved objects appear twice, because a single homography describes one rigid plane (or one rotation about the camera centre), not independently moving objects.*

This is the planar-scene assumption made explicit. A homography is exact for a planar scene seen from anywhere, or any scene seen from a camera that only rotates. Depth relief and moving objects become outliers, and the warp will show them as ghosts or double edges. For stitching this means the pair should be taken from a fixed viewpoint or of a distant or flat scene, and that the inlier count relative to the match count is a useful health signal.

## 10. Multi-attempt estimation

`ExtractAndMatchAndFindHomography_Multi` (src/cusift.cu:543) and its `_AndWarp` sibling run the RANSAC-plus-refinement step `num_homography_attempts` times on the same matches, with seed_ + k for attempt k (or a fresh random_device value when seed_ is 0), and keep one candidate by a selectable goal:

- `CUSIFT_HOMOGRAPHY_GOAL_MAX_INLIERS`: the candidate with the largest RANSAC inlier count;
- `CUSIFT_HOMOGRAPHY_GOAL_MIN_EYE_DIFF`: the candidate whose upper-left 2×2 block is closest to the identity in squared Frobenius norm, computed by `EyeDiff2x2`. This prefers the least rotation, scale and shear and is meant for setups where the motion is known to be almost pure translation.

Candidates that come out non-finite are skipped; if every attempt is non-finite the call fails with "Homography contains non-finite values in all attempts". Each surviving candidate is refined before it is judged, so the comparison is between refined homographies.

# 11. Failure modes and guards

| Condition | Where | Behaviour |
|---|---|---|
| fewer than 8 matched points, or fewer than 8 candidates after score/ambiguity filtering | `FindHomography_private` | H is left NaN with h22 = 1; the public wrapper reports "Homography contains non-finite values" |
| `num_loops_` outside [1, 1 048 560] or `improve_num_loops_` < 0 | `ValidateHomographyOptions` | argument error before any GPU work |
| $\lvert\det H\rvert < 10^{-12}$ | `Invert3x3` | "Homography is too close to singular to invert" |
| canvas wider than 2·max(w1, w2) or taller than 2·max(h1, h2) | `WarpImages` | "Warping: warped image too large", nothing allocated |
| normal equations not positive definite during refinement | `solve_cholesky_8x8` | refinement stops early and keeps the last good estimate |
| AVX2 build running on a CPU without AVX2/FMA | `ImproveHomography` | warning on stderr; the RANSAC estimate is returned unrefined |
| non-planar scene, moving objects, too little overlap | (not an error) | low inlier ratio; ghosting in the overlap; large extrapolation error at canvas edges |

All errors are reported through the thread-local error state (`CusiftHadError`, `CusiftGetLastErrorString`); nothing throws across the C boundary, and the RAII guards release every device and host buffer on the way out.

# 12. Code map

| Step | Function | Location |
|---|---|---|
| brute-force matching, score and ambiguity | `FindMaxCorr10` | src/matching.cu:39 |
| match driver | `MatchSiftData_private` | src/matching.cu:719 |
| 4-point sampling (seeded) | `GenerateRandomSamples` | src/matching.cu:383 |
| 4-point DLT, one hypothesis per thread | `ComputeHomographies` | src/matching.cu:279 |
| inlier counting per hypothesis | `TestHomographies` | src/matching.cu:329 |
| RANSAC driver (homography) | `FindHomography_private` | src/matching.cu:408 |
| RANSAC driver (similarity) | `FindSimilarity_private` | src/matching.cu:609 |
| model dispatch | `FindGeometricModel` | src/cusift.cu:88 |
| public estimate + refine | `FindHomography` | src/cusift.cu:276 |
| IRLS refinement, Cholesky solve | `ImproveHomography` | src/geomFuncs.c:210 |
| 3×3 inverse by cofactors | `Invert3x3` | src/cusift.cu:685 |
| canvas size, origin and corners (shared helper) | `ComputeCanvasGeometry` | src/cusift.cu:723 |
| public canvas query, no warp | `ComputeWarpGeometry` | src/cusift.cu:1071 |
| CPU bilinear sampler | `SampleBilinear` | src/cusift.cu:797 |
| dual warp kernel | `WarpDualKernel` | src/cusift.cu:829 |
| canvas sizing + warp (host output) | `WarpImages` | src/cusift.cu:880 |
| canvas sizing + warp (device output) | `WarpImages_GPU` | src/cusift.cu:996 |
| fused pipeline, host output | `ExtractAndMatchAndFindHomographyAndWarp` | src/cusift.cu:1084 |
| fused pipeline, device output | `ExtractAndMatchAndFindHomographyAndWarp_GPU` | src/cusift.cu:1230 |
| multi-attempt selection | `ExtractAndMatchAndFindHomography_Multi` | src/cusift.cu:543 |
| identity-distance goal | `EyeDiff2x2` | src/cusift.cu:533 |
| option structs and API docs | `FindHomographyOptions_t` | include/cusift.h:374 |

Line numbers refer to commit 1c26eba.

# 13. Measured cost of each stage

Wall-clock time per call on the bundled 1920×1080 pair, RTX 4070, second run after a warm-up pass, measured from Python around the C calls (so including host-device copies and ctypes overhead). The C demo's option values were used: thresh_ 2.0, 10 000 RANSAC loops, 5 refinement rounds, seed 42.

| Stage | Time |
|---|---|
| SIFT extraction, image 1 | 15.7 ms |

| Stage | Time |
|---|---|
| SIFT extraction, image 2 | 17.0 ms |
| matching (4005 × 4243 descriptors) | 5.7 ms |
| RANSAC (10 000 hypotheses) + 5 refinement rounds | 1.3 ms |
| warp, GPU path, 1922×1081 canvas (uploads and downloads included) | 9.4 ms |
| warp, CPU path (OpenMP) | 22.2 ms |

The geometry stages are a small fraction of the whole; extraction dominates, and the GPU warp cost is mostly PCIe traffic, which the fused pipeline avoids on the input side.

# Appendix A. Minimal C usage of the stitch step

```
#include "cusift.h"

InitializeCudaSift();
Image_t in1 = {pixels1, w1, h1}, in2 = {pixels2, w2, h2};   /* float32, row-major, [0,255] */
SiftData s1 = {0}, s2 = {0};
float H[9]; int inliers = 0;
Image_t out1 = {0}, out2 = {0};

ExtractAndMatchAndFindHomographyAndWarp(&in1, &in2, &s1, &s2, H, &inliers,
                                        &extract_opts, &homog_opts, &out1, &out2);
if (CusiftHadError()) { /* read CusiftGetLastErrorString(); H may be unusable */ }

/* out1, out2: outW x outH canvases in the frame of image 1; NaN = no data.
   Canvas pixel (x, y) is image-1 pixel (x + g.origin_u, y + g.origin_v). */
WarpGeometry_t g;
ComputeWarpGeometry(H, w1, h1, w2, h2, &g);   /* same canvas: size, origin, corners of both images */

FreeImage(&out1); FreeImage(&out2);
DeleteSiftData(&s1); DeleteSiftData(&s2);
```

# Appendix B. The same step from Python

```
import numpy as np
from PIL import Image
from cusift import CuSift, ExtractOptions, HomographyOptions

sift = CuSift()                                  # finds build/Release/cusift.dll
a = np.asarray(Image.open("a.png").convert("L"), dtype=np.float32)
b = np.asarray(Image.open("b.png").convert("L"), dtype=np.float32)

kp_a, kp_b = sift.extract(a), sift.extract(b)
sift.match(kp_a, kp_b)                           # fills the match fields of kp_a
H, n_inliers = sift.find_homography(kp_a, options=HomographyOptions(seed=42))
w_a, w_b = sift.warp_images(a, b, H, use_gpu=True)

va, vb = ~np.isnan(w_a), ~np.isnan(w_b)
out = np.where(va & vb, 0.5 * (w_a + w_b), np.where(va, w_a, w_b))
Image.fromarray(np.nan_to_num(out).clip(0, 255).astype(np.uint8)).save("stitched.png")
```